CENG203 – Week 4

Stack ve Queue

Lab için tam çalışma rehberi

İçindekiler: Büyük resim · C++ ön bilgisi · Proje yapısı · Q1–Q10 (kavram + ezber kalıbı + doğrulanmış kod + a/b/c cevapları) · Sık hatalar · Ezber kartı

Kodların hepsi derlenip çalıştırıldı; çıktıları worksheet'teki "Expected output" ile aynı.

0. Nasıl çalışmalı?

1. **Önce Bölüm 1–3'ü oku** (kavramlar + C++ araçları). Kod ezberlemek, bunları bilmeden işe yaramaz.
2. Her soruda önce **"Ezber kalıbı"** kutusunu ezberle: kod aslında hep 3–5 adımlık küçük bir kalıp.
3. Sonra kağıda **kendi başına** yaz, kodu kapat. Sonra karşılaştır.
4. Son sayfadaki **Ezber kartı**, sınavdan önce son bakış için.

1. Büyük resim

1.1 Stack = LIFO

Last In, First Out: son giren ilk çıkar. Tabak yığını. Her şey **sadece üstte (top)** olur.

İşlem	Ne yapar	Hata durumu
`push(x)`	üste ekler	dolu (sadece dizi) → **overflow**
`pop(v)`	üsttekini çıkarır, değeri `v`'ye yazar	boş → **underflow**, `false`
`peek(v)`	üsttekini okur, **çıkarmaz**	boş → `false`
`isEmpty()`, `size()`	boş mu / kaç eleman	–

Hepsi **O(1)**: hep aynı uçta çalışırsın, hiçbir şeyi gezmen gerekmez.

1.2 Queue = FIFO

First In, First Out: ilk giren ilk çıkar. Market kuyruğu. Sona (**rear**) eklenir, baştan (**front**) çıkar.

```
front -> [2] [7] [5] [8] <- rear
          ^               ^
       dequeue         enqueue
```

1.3 Dört sınıf (hepsi worksheet'te)

	Linked list ile (Node)	Dizi ile (array)
Stack	`Stack`: `top` işaretçisi + `count`. Taşmaz.	`ArrayStack`: `items[5]` + `topIndex` (boşken −1). Taşabilir.
Queue	`Queue`: `front` + `rear` + `count`. Taşmaz.	`CircularQueue`: `items[5]` + `front` + `count`, indeks başa sarar (`%`).

Dersin 2 kuralı (sınavda yazılır)
Hata kuralı: sınıf fonksiyonu ekrana hata yazmaz; `false` döndürür (ve çıkış parametresine dokunmaz). Ne yazılacağına çağıran (`Week04.cpp`) karar verir. Bu yüzden `bool pop(int& value)`.
Bellek kuralı: düğümü listeden *ayırmak (unlink)* onu *serbest bırakmak* değildir. Her `pop/dequeue` çıkardığı düğümü `delete` etmeli, kalanları destructor siler. Dizi sürümünde `new` yok → `delete` yok, ama taşabilir.

2. C++ ön bilgisi (bu haftanın araçları)

2.1 İşaretçi ve `->`

`Node* top;` bir düğümün **adresini** tutar. `nullptr` = hiçbir yeri göstermiyor (boş). Gösterilen düğümün alanına `top->data`, `top->next` ile ulaşılır. Gezme kalıbı:

```
for (Node* cur = top; cur != nullptr; cur = cur->next)
    cout << cur->data;
```

2.2 `new` ve `delete`

`new Node` heap'te yer ayırır ve adresini verir; sen `delete` diyene kadar yaşar. Her `new`'e bir `delete`. Silme sırası: **değeri al → işaretçiyi ilerlet → delete**.

```
Node* temp = top;     // 1) silinecek düğümü ayır
top = top->next;      // 2) başı ilerlet (önce!)
delete temp;          // 3) sonra sil
```

> **Sıra hatası**
> Önce `delete top;` yazıp sonra `top = top->next;` yazarsan silinmiş belleği okursun.

2.3 Referans parametre `int&`

Fonksiyon çağıranın değişkenini doğrudan değiştirir. `bool pop(int& value)`: dönüş değeri başarı, `value` asıl veri. Çağıran: `int v; if (s.pop(v)) cout << v;`

2.4 Diğer araçlar

Konu	Bilgi
`char` ↔ `int`	Harf aslında sayı (ASCII): `'C'` = 67. `push('C')` → 67 saklanır. `cout << v` → **67**, `cout << (char)v` → **C**.
`%` (mod)	Bölümden kalan. `(4+1)%5 = 0` → 4'ten sonra 0'a sarar.
`static const int CAPACITY = 5;`	Sınıfın sabiti; nesnenin içinde yer kaplamaz.
Boyutlar (x64)	`int` =4, işaretçi=8, `Node` =**16** (4 + 4 boşluk + 8). `ArrayStack` nesnesi = 5×4 + 4 = **24** bayt.
`boolalpha`	`cout` true/false yazsın (1/0 değil).
Yıkıcı `~Stack()`	Nesne yok olurken otomatik çalışır; kalan düğümleri siler.

3. Proje yapısı ve Part A (kurulum)

```
CENG203_Lab\
  main.cpp                ← menüde "4. Week 4" + case 4: runWeek04();
  Labs\Labs.h             ← void runWeek04();
  Labs\Week04.cpp         ← test kodu + 4 yardımcı (reverseWithStack, isBalanced, TwoStackQueue, Editor)
  LinkedList\Node.h       ← Week 3'ten, aynen kullanılıyor
  Stack\Stack.h / .cpp          ← Q2
  Stack\ArrayStack.h / .cpp     ← Q3
  Queue\Queue.h / .cpp          ← Q5
  Queue\CircularQueue.h / .cpp  ← Q7
```

- **Tek solution, tek proje, tek main().** Yeni yapı = yeni klasör, yeni proje değil.
- Her `.cpp`'nin **ilk satırı kendi header'ı**: `#include "Stack.h"`.
- Header'lara yol: `Week04.cpp`'den `"../Stack/Stack.h"`, `main.cpp`'den `"Labs/Labs.h"`.
- Gövdeler sadece `.cpp`'de; header'da sadece prototip.
- Derle **Ctrl+Shift+B** (Output: "1 succeeded"), çalıştır **Ctrl+F5**, menüde **4**, sonra **0**.
- Stub'larla çıktı: `top ->`, `false`, `0`. Sonra Q2–Q9'da doldurulur.
- Lab sonu: `Git → Commit All and Push`, mesaj `Week 4 - stacks and queues`.

> **Çıktıdaki −858993460 değeri nedir?**
> Fonksiyon boşken `v`'ye yazmaz; Debug modu başlatılmamış yığın belleğini `0xCC` ile doldurur → `0xCCCCCCCC` = −858993460. Hata değil, stub'ın işareti.

Q1 – Stack kavramları (5 puan)

a) Prensip: LIFO – Last In, First Out. Son konan eleman ilk alınır; ekleme ve çıkarma hep aynı uçtan (üstten) yapılır, tabak yığını gibi.

b) İz: push(2), push(7), push(5), push(8), pop(), push(4)

Adım	Stack (üst solda)
push(2)	2
push(7)	7, 2
push(5)	5, 7, 2
push(8)	8, 5, 7, 2
pop() → 8	5, 7, 2
push(4)	4, 5, 7, 2

```
push(8)         pop() -> 8       push(4)
top -> [ 8 ]    top -> [ 5 ]     top -> [ 4 ]
       [ 5 ]           [ 7 ]            [ 5 ]
       [ 7 ]           [ 2 ]            [ 7 ]
       [ 2 ]                            [ 2 ]
```

Son üç adım çizilir (her birinde en üst kutunun yanına **top** yaz).

c) Overflow: dolu stack'e push. **Underflow:** boş stack'ten pop. **ArrayStack** taşabilir çünkü kapasite sabit (5). **Linked Stack** taşmaz çünkü her push'ta `new` ile yeni düğüm açılır (bellek bitene kadar).

d) `peek()` üsttekini **okur, çıkarmaz** (stack değişmez); `pop()` üsttekini **çıkarır** ve verir (stack küçülür). Üçü de O(1): sadece en üst elemanla çalışır, eleman sayısı arttıkça iş artmaz.

Q2 – Linked-list Stack (10 puan)

Ezber kalıbı

push = Week 3 `insertAtFront`: `new → data → next=top → top=new → count++`
pop = boşsa `false`; `value=top->data`; `temp=top; top=top->next; delete temp;`; `count--`; `true`
peek = pop'un silmeyen hali
isEmpty = `top == nullptr` **clear** = `while(pop(v));`

```
#include "Stack.h"
#include <iostream>
using namespace std;

Stack::Stack()
{
    top = nullptr;
    count = 0;
}

Stack::~Stack()
{
    clear();
}

void Stack::push(int value)
{
    Node* newNode = new Node;       // 1) allocate
    newNode->data = value;          // 2) set data
    newNode->next = top;            // 3) point to the old top
    top = newNode;                  // 4) new node becomes the top
    count++;
}

bool Stack::pop(int& value)
{
    if (top == nullptr)             // underflow
        return false;
    value = top->data;              // give the value back
    Node* temp = top;
    top = top->next;                // move top down
    delete temp;                    // free the node
    count--;
    return true;
}

bool Stack::peek(int& value) const
{
    if (top == nullptr)
        return false;
    value = top->data;              // read only: nothing is removed
    return true;
}

bool Stack::isEmpty() const
{
    return top == nullptr;
}

int Stack::size() const
{
    return count;
}

void Stack::clear()
{
    int value;
    while (pop(value))
        ;
}

void Stack::display() const
{
    cout << "top ->";
    if (top == nullptr)
        cout << " (empty)";
```

```
    for (Node* cur = top; cur != nullptr; cur = cur->next)
        cout << " " << cur->data;
    cout << endl;
}
```

Beklenen çıktı

```
[Q2] top -> 8 5 7 2
[Q2] pop -> 8
[Q2] top -> 4 5 7 2
[Q2] peek -> 4  size = 4
[Q2] pop -> 4 / 5 / 7 / 2   (hepsi ayrı satır)
[Q2] pop on empty = false
```

a) Neden top = listenin başı? Başa ekleme/silme O(1) (gezme yok). Son düğüm olsaydı, silmek için bir öncekine ulaşmak gerekir → tek yönlü listede O(n).

b) −1 neyi bozar? −1 de geçerli bir veri olabilir; çağıran "stack boş" mu yoksa "−1 saklanmış" mı ayırt edemez. `bool` + `int&` hata ile veriyi ayırır.

Try it – c) `delete temp;` satırı silininces düğüm artık `top` ile ulaşılamaz ama bellekte durur → **bellek sızıntısı (memory leak)**. Düğümü listeden ayırmak (unlink) belleği serbest bırakmaz; her `pop`'ta `delete` şart. Diagnostic Tools'ta Process Memory sürekli artar.

Q3 – Array Stack (10 puan)

Ezber kalıbı

`topIndex == -1` boş | `topIndex == CAPACITY-1` dolu | `size = topIndex+1`

push: doluysa `false`; `topIndex++`; `items[topIndex]=value` (**önce** artır, **sonra** yaz)

pop: boşsa `false`; `value=items[topIndex]`; `topIndex--` (**önce** oku, **sonra** azalt)

display: `for (i = topIndex; i >= 0; i--)` (azalan)

```
#include "ArrayStack.h"
#include <iostream>
using namespace std;

ArrayStack::ArrayStack()
{
    topIndex = -1;
}

bool ArrayStack::push(int value)
{
    if (isFull())                   // overflow
        return false;
    topIndex++;
    items[topIndex] = value;
    return true;
}

bool ArrayStack::pop(int& value)
{
    if (isEmpty())                  // underflow
        return false;
    value = items[topIndex];
    topIndex--;                     // the old value stays in the array
    return true;
}

bool ArrayStack::peek(int& value) const
{
    if (isEmpty())
        return false;
    value = items[topIndex];
    return true;
}

bool ArrayStack::isEmpty() const
{
    return topIndex == -1;
}

bool ArrayStack::isFull() const
{
    return topIndex == CAPACITY - 1;
}

int ArrayStack::size() const
{
    return topIndex + 1;
}

void ArrayStack::display() const
{
    cout << "top ->";
    if (isEmpty())
        cout << " (empty)";
    for (int i = topIndex; i >= 0; i--)
        cout << " " << items[i];
    cout << endl;
}
```

Beklenen çıktı

```
push(10..50) = true (5 satır)   push(60) = false
top -> 50 40 30 20 10
isFull = true  size = 5
pop -> 50  push(60) = true
top -> 60 40 30 20 10
```

a) `isEmpty(): topIndex == -1` `isFull(): topIndex == CAPACITY - 1` (yani 4).

b) isFull kontrolü kaldırılınca: 6. push `items[5]` 'e yazar = dizi dışı (**buffer overflow**), komşu belleği ezer. Visual Studio Debug'da genelde "*Run-Time Check Failure #2 – Stack around the variable … was corrupted*" verir. Linked stack ise sadece büyür; dizi sessizce belleği bozabilir (tanımsız davranış), bu daha tehlikeli.

c) Hayır, pop değeri silmez. Sadece `topIndex` bir azalır; eski değer dizide kalır, sonraki push üstüne yazar.

d) Bellek (x64): ArrayStack = 5×4 + 4 (topIndex) = **24 bayt**, sabit. Linked Stack'te 5 değer = 5 düğüm × 16 = 80 + nesne (`top` 8 + `count` 4 + boşluk 4 = 16) = **96 bayt**. **Dizi:** en fazla eleman sayısı biliniyorsa, hızlı ve az bellek. **Liste:** eleman sayısı bilinmiyorsa, taşmasın istiyorsan.

Q4 – String'i stack ile ters çevir (10 puan)

Ezber kalıbı

`Stack st;` → her harf `st.push(c)` → `while (st.pop(v)) result += (char)v;`

```
string reverseWithStack(const string& s)
{
    string result;
    Stack st;
    for (char c : s)
        st.push(c);                 // char converts to int
    int v;
    while (st.pop(v))
        result += (char)v;          // back to char
    return result;
}
```

```
[Q4] 302GNEC
[Q4] kcats
```

a) Stack LIFO: en son push edilen harf ilk pop edilir; sıra tersine döner.

b) `push('C')` düğümde **67** (C'nin ASCII kodu) saklar. `cout << value` → **67**, `cout << (char)value` → **C**.

c) n harf için zaman **O(n)** (n push + n pop), ek bellek **O(n)** (stack'te n düğüm).

d) Bonus: `bool isPalindrome(const string& s) { return s == reverseWithStack(s); }` → `"level"` true, `"stack"` false.

Q5 – Linked-list Queue (10 puan)

Ezber kalıbı

enqueue: yeni düğüm (`next=nullptr`); **kuyruk boşsa** `front=rear=yeni` ; değilse `rear->next=yeni; rear=yeni;` ; `count++`

dequeue: boşsa `false` ; `value=front->data; temp=front; front=front->next;` **if (front==nullptr) rear=nullptr;** `delete temp; count--;`

EN ÖNEMLİ: kuyruk boşalınca `rear` da `nullptr` olmalı.

```
#include "Queue.h"
#include <iostream>
using namespace std;

Queue::Queue()
{
    front = nullptr;
    rear = nullptr;
    count = 0;
}

Queue::~Queue()
{
    int value;
    while (dequeue(value))          // dequeue until the queue is empty
        ;
}

void Queue::enqueue(int value)
{
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;        // the new node is the last one
    if (rear == nullptr) {          // empty queue: front and rear are the same node
        front = newNode;
        rear = newNode;
    } else {
        rear->next = newNode;       // link behind the old rear
        rear = newNode;
    }
    count++;
}

bool Queue::dequeue(int& value)
{
    if (front == nullptr)           // underflow
        return false;
    value = front->data;
    Node* temp = front;
    front = front->next;
    if (front == nullptr)           // queue became empty: rear must be reset too
        rear = nullptr;
    delete temp;
    count--;
    return true;
}

bool Queue::peek(int& value) const
{
    if (front == nullptr)
        return false;
    value = front->data;
    return true;
}

bool Queue::isEmpty() const
{
    return front == nullptr;
}

int Queue::size() const
{
    return count;
}

void Queue::display() const
```

```
{
    cout << "front ->";
    if (front == nullptr)
        cout << " (empty)";
    for (Node* cur = front; cur != nullptr; cur = cur->next)
        cout << " " << cur->data;
    cout << " <- rear" << endl;
}
```

```
[Q5] front -> 2 7 5 8 <- rear
[Q5] dequeue -> 2
[Q5] front -> 7 5 8 4 <- rear
[Q5] peek -> 7  size = 4
[Q5] dequeue -> 7 / 5 / 8 / 4
[Q5] dequeue on empty = false
[Q5] front -> 9 <- rear
```

a) `rear` son düğümü gösterir; enqueue sona O(1) eklenir. Olmasa her enqueue'da son düğüme kadar gezmek gerekir → **O(n)**.

Try it – b) `rear = nullptr` satırları silinince son eleman çıkınca `front` `nullptr` olur ama `rear` **silinmiş düğümü gösterir** (dangling pointer). `enqueue(9)` `rear != nullptr` sanıp `rear->next`'e (serbest bırakılmış belleğe) yazar; `front` hâlâ `nullptr` kaldığı için kuyruk boş görünür (veya program çöker). Yanlış olan pointer: **rear**.

c) Baştan çıkarmak O(1) (`front->next` yeni baş). Sondan çıkarmak için son düğümün **bir öncekini** bulmak gerekir (tek yönlü liste, geri link yok) → O(n). Ayrıca FIFO tanımı: ilk giren ilk çıkar.

Q6 – Parantez kontrolü (10 puan)

Ezber kalıbı

açılış `( [ {` → **push** kapanış `) ] }` → **pop** (pop başarısızsa `false`) ve **türü eşleşiyor mu** bak (eşleşmezse `false`) sonda **stack boş mu** (`return st.isEmpty();`)

```
bool isBalanced(const string& s)
{
    Stack st;
    int v;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);                         // opening: remember it
        } else if (c == ')' || c == ']' || c == '}') {
            if (!st.pop(v))                     // closing but nothing open
                return false;
            char open = (char)v;
            if ((c == ')' && open != '(') ||
                (c == ']' && open != '[') ||
                (c == '}' && open != '{'))
                return false;                   // wrong type
        }
    }
    return st.isEmpty();                        // anything left open = unbalanced
}
```

```
"([]{})" -> true   "{[()()]}" -> true   "([)]" -> false
"((" -> false       ")(" -> false         "" -> true
```

a) Sayaç sadece **sayıyı** tutar, **türü ve sırayı** tutmaz. `"([)]"`'de +1 +1 −1 −1 = 0 çıkar ama parantezler yanlış iç içe: `)` geldiğinde üstte `[` var, eşleşmez.

b) Son karakterden sonra stack boş değilse: **kapatılmamış açılış parantezi** var (`"(("`). Ortada pop başarısızsa: **eşi olmayan kapanış** var (`")("`).

c) `"{[()]}"` **izi** (stack üstten alta):

Karakter	İşlem	Stack
`{`	push	{
`[`	push	[{
`(`	push	([{
`)`	pop (eşleşti	[{
`]`	pop [eşleşti	{
`}`	pop { eşleşti	boş → **true**

Q7 – Circular Queue (15 puan)

Ezber kalıbı: 3 formül

sıradaki boş yer: `(front + count) % CAPACITY`

dequeue sonrası: `front = (front + 1) % CAPACITY`

boş: `count == 0` **dolu:** `count == CAPACITY`

enqueue: doluysa `false`; `items[(front+count)%CAPACITY]=value; count++;`

dequeue: boşsa `false`; `value=items[front]; front=(front+1)%CAPACITY; count--;`

```
#include "CircularQueue.h"
#include <iostream>
using namespace std;

CircularQueue::CircularQueue()
{
    front = 0;
    count = 0;
    for (int i = 0; i < CAPACITY; i++)
        items[i] = 0;
}

bool CircularQueue::enqueue(int value)
{
    if (isFull())                               // overflow
        return false;
    items[(front + count) % CAPACITY] = value;  // next free slot, wraps around
    count++;
    return true;
}

bool CircularQueue::dequeue(int& value)
{
    if (isEmpty())                              // underflow
        return false;
    value = items[front];
    front = (front + 1) % CAPACITY;             // wraps from 4 back to 0
    count--;
    return true;
}

bool CircularQueue::peek(int& value) const
{
    if (isEmpty())
        return false;
    value = items[front];
    return true;
}

bool CircularQueue::isEmpty() const
{
    return count == 0;
}

bool CircularQueue::isFull() const
{
    return count == CAPACITY;
}

int CircularQueue::size() const
{
    return count;
}

void CircularQueue::display() const
{
    cout << "front ->";
    for (int i = 0; i < count; i++)
        cout << " " << items[(front + i) % CAPACITY];
    if (isEmpty())
        cout << " (empty)";
    cout << " <- rear   | array:";
    for (int i = 0; i < CAPACITY; i++)
        cout << " " << items[i];
```

```
    cout << "  front=" << front << " count=" << count << endl;
}
```

```
enqueue(1..5) = true   enqueue(6) = false
front -> 1 2 3 4 5 <- rear   | array: 1 2 3 4 5  front=0 count=5
dequeue -> 1     dequeue -> 2
enqueue(6) = true  enqueue(7) = true
front -> 3 4 5 6 7 <- rear   | array: 6 7 3 4 5  front=2 count=5
```

a) İki dequeue sonrası `front=2`, `count=3`. `enqueue(6)` → `(2+3)%5 = 0` → slot 0. `enqueue(7)` → `(2+4)%5 = 1` → slot 1. FIFO sırası front'tan başlar: slot 2,3,4,0,1 = 3 4 5 6 7; ham dizi: 6 7 3 4 5.

b) Düz dizi kuyrukta `front` sadece artar; baştaki boşalan yerlere bir daha yazılamaz, dizinin sonu dolunca kuyruk "dolu" der (başta boş yer olduğu halde). Ya her seferinde kaydırırsın (O(n)) ya da yer israf olur. Circular queue **baştaki boş yerleri yeniden kullanır**.

c) Sadece `front` ve `rear` ile dolu ve boş durumda ikisinde de `front == rear` olur; ayırt edilemez. İki yaygın çözüm: (1) **sayaç (count)** tutmak (bizim yaptığımız), (2) **bir slotu hep boş bırakmak** (dolu: `(rear+1)%N == front`). Üçüncüsü bayrak (flag) tutmak.

d) Klavye tamponu, ses/video akış tamponu, ağ paket tamponu, yazıcı tamponu, round-robin sıra.

Q8 – İki stack ile Queue (10 puan)

Ezber kalıbı

enqueue = `inbox.push(value)` | **dequeue** = `if (outbox.isEmpty())` {inbox'tan hepsini `pop` → `outbox.push`}; sonra `return outbox.pop(value);` | **isEmpty** = ikisi de boş

```
class TwoStackQueue
{
private:
    Stack inbox;                    // enqueue pushes here
    Stack outbox;                   // dequeue pops from here

public:
    void enqueue(int value)
    {
        inbox.push(value);
    }

    bool dequeue(int& value)
    {
        if (outbox.isEmpty()) {         // refill: this reverses the order
            int moved;
            while (inbox.pop(moved))
                outbox.push(moved);
        }
        return outbox.pop(value);
    }

    bool isEmpty() const
    {
        return inbox.isEmpty() && outbox.isEmpty();
    }
};
```

```
[Q8] dequeue -> 1 / 2 / 3 / 4      [Q8] isEmpty = true
```

a) 1, 2, 3 enqueue → inbox (üstten alta): **3 2 1**. dequeue'da outbox boş → taşı: 3 çıkar-outbox'a, 2 çıkar-outbox'a, 1 çıkar-outbox'a → outbox (üstten alta): **1 2 3**. Pop → **1**. Stack'ten stack'e aktarmak sırayı **ters çevirir**; ters sıranın tersi doğru sıra (FIFO) olur.

b) En kötü dequeue: outbox boşken n eleman taşınır → **O(n)**. Ama her eleman en fazla bir kez inbox→outbox taşınır ve bir kez pop edilir; toplam iş eleman başına sabit → **amortize O(1)**.

c) Composition (bileşim): bir sınıfı başka sınıfın içinde **üye olarak kullanmak** (TwoStackQueue'nun içinde iki Stack). İyi tasarım: hazır, test edilmiş sınıf yeniden kullanılır, Stack'e dokunulmaz, kod tekrarı olmaz.

Q9 – Undo / Redo (10 puan)

Ezber kalıbı

type(c): `text+=c; undoStack.push(c); redoStack.clear();`
undo: `undoStack.pop(v)` başarısızsa `false`; `text.pop_back(); redoStack.push(v); true`
redo: `redoStack.pop(v)` başarısızsa `false`; `text += (char)v; undoStack.push(v); true`

```
class Editor
{
private:
    string text;
    Stack undoStack;                // characters typed, newest on top
    Stack redoStack;                // characters undone, newest on top

public:
    void type(char c)
    {
        text += c;
        undoStack.push(c);          // remember for undo
        redoStack.clear();          // a new action kills the redo history
    }

    bool undo()
    {
        int v;
        if (!undoStack.pop(v))
            return false;
        text.pop_back();            // remove the last character
        redoStack.push(v);
        return true;
    }

    bool redo()
    {
        int v;
        if (!redoStack.pop(v))
            return false;
        text += (char)v;            // put the character back
        undoStack.push(v);
        return true;
    }

    string getText() const
    {
        return text;
    }
};
```

```
[Q9] typed abc     -> "abc"
[Q9] undo, undo    -> "a"
[Q9] redo          -> "ab"
[Q9] type x        -> "abx"
[Q9] redo after type = false
```

a) Undo "en son yapılan ilk geri alınır" ister = LIFO → stack. Geri alınanları saklamak için ikinci stack gerekir (redo). Tek stack'te yapılanlar ve geri alınanlar karışır. **Queue** en eskiyi ilk verir → yanlış sıra (son yazılan harf değil ilk yazılan silinirdi).

b) Geri alıp yeni bir şey yazınca eski "gelecek" artık geçerli değil. Silinmezse `type('x')`'ten sonra `redo`, redoStack'te kalan `'c'`'yi geri getirir → **"abxc"** olurdu (anlamsız).

c) Tarayıcıda **Back yığını = undoStack** (gidilen sayfalar), **Forward yığını = redoStack** (Back ile geri dönülen sayfalar). Yeni bir sayfaya gidince Forward yığını silinir, tıpkı type'ta redo'nun silinmesi gibi.

Q10 – Hangi yapı, ne kadar hızlı? (10 puan)

a) Tablo

	Stack (linked)	ArrayStack	Queue (linked)	CircularQueue
ekle (push / enqueue)	O(1)	O(1)	O(1)	O(1)
çıkar (pop / dequeue)	O(1)	O(1)	O(1)	O(1)
taşabilir mi?	hayır	evet	hayır	evet
çalışırken büyür mü?	evet	hayır	evet	hayır
bir eleman için bellek	16 bayt	4 bayt	16 bayt	4 bayt

Linked Queue'nun enqueue'su `rear` sayesinde O(1). Linked düğüm = 4 (data) + 4 (hizalama boşluğu) + 8 (next) = 16.

b) Stack mi, Queue mu?

Durum	Cevap	Neden
(1) Yazıcı kuyruğu (spooler)	Queue	ilk gelen iş ilk basılır
(2) Word'de Undo	Stack	son yapılan ilk geri alınır
(3) Çalışan programın fonksiyon çağrıları	Stack	son çağrılan ilk döner (call stack)
(4) Labirentte BFS	Queue	önce yakın düğümler (katman katman)
(5) Derleyicide parantez eşleme	Stack	son açılan ilk kapanır
(6) Round-robin CPU zamanlaması	Queue	sırayla döner, ilk gelen ilk çalışır

c) Priority queue: elemanlar **geliş sırasına** göre değil **önceliğe** göre çıkar (en yüksek öncelikli önce). Bugünkü queue sadece FIFO. (Genelde heap ile yapılır.)

d) Hata nasıl döndürülür? Üç yol: (1) **özel değer** döndürmek (örn. −1) — veri de −1 olabilir, belirsiz; (2) **bool döndür + sonucu referans parametreyle ver** (`bool pop(int& value)`) — bu derste kullanılan; (3) **exception fırlatmak** (`throw`). **Bu ders (2)'yi kullanır:** hata ile veri karışmaz, fonksiyon ekrana yazmaz, ne yapılacağına çağıran karar verir.

e) Kendi GitHub repo URL'ni yaz (`https://github.com/<kullanıcı>/CENG203_Lab`).

4. Sık hatalar (worksheet A.1)

Gördüğün	Sebep → çözüm
"Unable to start program … .exe, cannot find the file"	Derleme başarısız / hiç derlenmedi → "build errors, continue?" sorusuna **No**; Error List'te ilk hatayı düzelt, Ctrl+Shift+B, "1 succeeded" gör.
Local Windows Debugger yok, "Select Startup Item"	Folder View'dasın → Switch Views → `CENG203_Lab.sln`.
LNK2019 unresolved external symbol runWeek04	Dosya projede değil veya gövde yok/isim yanlış → Show All Files → Include In Project → Save All → build.
LNK2005 already defined	Aynı fonksiyonun iki gövdesi / gövde header'da → tek main, gövdeler sadece .cpp'de.
C1083 Cannot open include file 'Stack.h'	Yol yanlış → `"../Stack/Stack.h"`.
Labs, Stack, Queue klasörleri `CENG203_Lab\CENG203_Lab\` içinde	Build klasörüne eklemişsin → File Explorer'da `.vcxproj` ve `main.cpp` yanına taşı, Include In Project.
Ctrl+F7 program üretmiyor	.cpp tek başına program değil → Ctrl+Shift+B, sonra Ctrl+F5.
C2653 / C2065 'Stack'	.cpp kendi header'ını include etmemiş → ilk satıra `#include "Stack.h"`.
LNK1168 cannot open .exe for writing	Önceki konsol penceresi açık → kapat, tekrar derle.
Eski sürüm gibi davranıyor	Build → Rebuild Solution; sekmenin üstüne gelip dosya yolunu kontrol et.

5. Ezber kartı (sınav öncesi son bakış)

```
STACK (top, count)
push : n=new Node; n->data=v;
       n->next=top; top=n; count++;
pop  : if(top==nullptr) return false;
       v=top->data; t=top;
       top=top->next; delete t;
       count--; return true;
peek : pop gibi, silme yok
isEmpty: top==nullptr
clear: while(pop(v));
display: cur=top -> cur->next
```

```
ARRAYSTACK (topIndex=-1)
boş : topIndex==-1
dolu: topIndex==CAPACITY-1
size: topIndex+1
push: dolu->false; items[++topIndex]=v
pop : boş->false; v=items[topIndex--]
display: i=topIndex ... 0 (azalan)
```

```
REVERSE
Stack st; her c: st.push(c);
while(st.pop(v)) result+=(char)v;
```

```
BALANCED
açılış  -> push
kapanış -> pop (olmazsa false)
           türü uymazsa false
sonda: return st.isEmpty();
```

```
QUEUE (front, rear, count)
enqueue: n=new; n->next=nullptr;
  if(rear==nullptr) front=rear=n;
  else { rear->next=n; rear=n; }
  count++;
dequeue: if(front==nullptr) false;
  v=front->data; t=front;
  front=front->next;
  if(front==nullptr) rear=nullptr;  !!
  delete t; count--; true;
```

```
CIRCULAR (front=0, count=0)
enq : dolu->false;
      items[(front+count)%CAP]=v;
      count++;
deq : boş->false; v=items[front];
      front=(front+1)%CAP; count--;
boş: count==0  dolu: count==CAP
```

```
TWO-STACK QUEUE
enqueue: inbox.push(v)
dequeue: if(outbox.isEmpty())
           while(inbox.pop(t)) outbox.push(t);
         return outbox.pop(v);
isEmpty: inbox ve outbox boş
```

```
EDITOR
type: text+=c; undo.push(c); redo.clear();
undo: undo.pop(v); text.pop_back(); redo.push(v);
redo: redo.pop(v); text+=(char)v; undo.push(v);
```

Tek cümlelik özetler

Stack LIFO, Queue FIFO. Hata = `false` döndür, ekrana yazma. `pop/dequeue` her zaman `delete`. Queue boşalınca `rear = nullptr`. Circular: `%` ile sar, `(front+count)%CAP`. Parantez: açılış push, kapanış pop+eşleştir, sonda boş. Undo/redo: iki stack, `type` redo'yu siler.